OPEN
Compute Project

Hardware Fault Management Project

System Debug Specification

0.1

Modular Base Specification
Effective July 21, 2026

Author: (See Acknowledgements Section)

Current Template Version:

3 Layer (Base, Design and Product) Specification Template V1.6.0

Table of Contents

List of Figures

List of Tables

1 License

Open Web Foundation (OWF) CLA

Contributions to this Specification are made under the terms and conditions set forth in **Modified Open Web Foundation Agreement 0.9 (OWFa 0.9)**. (As of October 16, 2024) (“Contribution License”) by:

- TODO: fill in

Usage of this Specification is governed by the terms and conditions set forth in **Modified OWFa 0.9 Final Specification Agreement (FSA)** (As of October 16, 2024) **(“Specification License”)**.

You can review the applicable Specification License(s) referenced above by the contributors to this Specification on the OCP website at https://www.opencompute.org/contributions/templates-agreements.

For actual executed copies of either agreement, please contact OCP directly.

NOTWITHSTANDING THE FOREGOING LICENSES, THIS SPECIFICATION IS PROVIDED BY OCP “AS IS” AND OCP EXPRESSLY DISCLAIMS ANY WARRANTIES (EXPRESS, IMPLIED, OR OTHERWISE), INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, OR TITLE, RELATED TO THE SPECIFICATION. NOTICE IS HEREBY GIVEN, THAT OTHER RIGHTS NOT GRANTED AS SET FORTH ABOVE, INCLUDING WITHOUT LIMITATION, RIGHTS OF THIRD PARTIES WHO DID NOT EXECUTE THE ABOVE LICENSES, MAY BE IMPLICATED BY THE IMPLEMENTATION OF OR COMPLIANCE WITH THIS SPECIFICATION. OCP IS NOT RESPONSIBLE FOR IDENTIFYING RIGHTS FOR WHICH A LICENSE MAY BE REQUIRED IN ORDER TO IMPLEMENT THIS SPECIFICATION. THE ENTIRE RISK AS TO IMPLEMENTING OR OTHERWISE USING THE SPECIFICATION IS ASSUMED BY YOU. IN NO EVENT WILL OCP BE LIABLE TO YOU FOR ANY MONETARY DAMAGES WITH RESPECT TO ANY CLAIMS RELATED TO, OR ARISING OUT OF YOUR USE OF THIS SPECIFICATION, INCLUDING BUT NOT LIMITED TO ANY LIABILITY FOR LOST PROFITS OR ANY CONSEQUENTIAL, INCIDENTAL, INDIRECT, SPECIAL OR PUNITIVE DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS SPECIFICATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND EVEN IF OCP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

2 Acknowledgements

The contributors of this Specification would like to acknowledge the following:

- Marko Bartscherer (Intel)
- Enrico Carrieri (Qualcomm Technologies, Inc.)
- Donald Gaither (Vertiv)
- James Zou (Aivres)

Also, we wish to acknowledge Chris Fenner (Google) and the Trusted Computing Group for the use of their Pandoc markdown tooling.

3 Revision History

Rev.	Date	Guiding Contributor(s)	Description
0.1	2026-05-04	Marko B., Enrico C., Donald G., James Z.	Initial Release

4 Compliance with OCP Tenets

4.1 Openness

This Specification is open.

4.2 Efficiency

This Specification is efficient.

4.3 Impact

This Specification is impactful.

4.4 Scale

This Specification is scalable.

4.5 Sustainability

This Specification is sustainable.

5 Introduction

With the rise of large datacenter and AI systems, nodes and components are becoming more diverse and increasing in size and complexity. For example, training LLM models in datacenters requires orchestration of clusters consisting of a large number of server systems. The scale of such clusters is determined by factors including model size (parameters) and training dataset volume. During long-duration training, system failures are expected and can result in workload interruption, reduced efficiency, increased operational cost and delayed new AI version releasing.

This increasing complexity requires the need to validate, diagnose, debug, and optimize beyond the component level. Before the publication of this Specification, the datacenter community was lacking a standardized methodology to extract diagnostic data and perform debug operations across the datacenter at the different levels from components to cluster. There was a need to develop a Specification that defines how debug data is exchanged between all components across the datacenter and the debug tooling. The community desired to standardize the necessary control and transport protocols enabling uniform debug of diverse systems.

The following challenges are commonly observed in data center debugging environments: It is difficult to identify the failure root cause when a training job terminates unexpectedly. Normally it takes multiple iterations to isolate and resolve hardware-related issues. Additional hardware failures may occur after the hardware issue was resolved. Sometimes, it is difficult to distinguish between hardware-induced and software-induced failures.

System designs that comply with OCP Fault Management Infrastructure Requirements [1] and OCP GPU & Accelerator RAS Requirements [2] improve observability and reliability. However, such compliance does not eliminate system failures. Efficient system-level debugging remains a critical requirement for data center operations.

This Specification will address debugging components and nodes within a datacenter across the lifecycle of the datacenter. It will prescribe an architecture to communicate various debug data in a common method and data models across the datacenter. The Specification will define interfaces, protocols, APIs, and data models incorporating existing work from other OCP Workstreams and other standards bodies improving the effectiveness of these for debugging in the datacenter. A system-level debugging framework is created to improve fault observability, root cause analysis (RCA), and remediation efficiency in server systems deployed in datacenters. Scalability across the lifecycle of the datacenter including but not limited to accessible ports, diagnostic and debug capabilities, and security and privacy requirements.

5.1 Goals

The main goal of this Specification is to prescribe an architecture, including interfaces, protocols, APIs, and data models, that communicates debug data across the datacenter. This will be done by:

- Defining the use of common connectivity, transports, and protocols.
  - Define a common communication “pipe” for debug data/messages.
  - Data/message payloads stay vendor specific.
- Using existing specifications/standards when available.
  - Leverage existing standard transports (e.g., USB) and protocols (e.g., PLDM)
- Allowing flexibility but provide interoperability.

  - Flexibility in how the system is put together.
  - Debug capabilities need to be discoverable and self describing
  - Interoperability with both different components and different debug tooling.
  - Data needs to be opaque to components in the middle (e.g., BMC).
- Providing scalability across the life cycle of development.
  - Use starting from manufacturing through to the at-scale, in-field deployment.

5.2 Document Scope

5.2.1 In Scope

5.2.2 Out of Scope

This Specification will not cover capabilities related to performance monitoring and telemetry.

In addition, this Specification will not cover existing material already covered by another OCP specification even if it is related to the goals stated in Section 5.1. This includes the following:

- Error Events - covered by the RAS API Specification [3].
- Bulk Dumping - covered by the Hyperscale CPU Debug and RAS Requirements [4] and GPU & Accelerator RAS Requirements [2] documents.
- Crash Dumping - covered by the RAS API Specification [3], Hyperscale CPU Debug and RAS Requirements [4], and GPU & Accelerator RAS Requirements [2] documents.
- Register Dumping - covered by the Hyperscale CPU Debug and RAS Requirements [4] and GPU & Accelerator RAS Requirements [2] documents.
- Array Dumping - covered by the GPU & Accelerator RAS Requirements [2] document.

5.3 Audience

5.4 Objectives

This Specification defines requirements in the following areas: standardization of failure reporting; definition of debug levels based on system complexity; provision of security controls for debug data and support for vendor-specific diagnostic extensions.

5.4.1 Standardization of Failure Reporting

The system shall standardize: failure report items, data formats, data collection methods and reporting sequences. All debug data shall be represented using structured JSON key-value pairs to ensure consistency and machine readability. The system should support access to debug data via industry-standard interfaces such as Redfish. Each reported event shall include timestamp information sufficient to enable event ordering and cross-component correlation.

5.4.2 Debug Levels

During the development of the server system, we can define which level of debugging the system will support. Low debug level provides minimal diagnostic information (e.g., error codes and basic status indicators). High debug level provides detailed diagnostic information, including but not limited to error trigger conditions, fault location, timing information, error counters and history. The supported debug level shall be configurable based on debugging bus bandwidth, debugging device complexity and debugging device storage volume.

5.4.3 Security Controls

The system shall provide mechanisms to protect sensitive debug data. Security features shall include: configurable access control for debug data, optional encoding or protection of configuration and status register contents, integration with platform-level security configuration (e.g., BIOS settings). Debug data collection, storage, and access mechanisms shall comply with the configured security policies.

5.4.4 Vendor-Specific Extensions

The system shall support vendor-specific extensions to enable additional diagnostic capabilities. Standardized data structures shall be preserved for interoperability. Vendor-specific data fields may be included as extensions to the standard schema. The system shall support access to vendor-specific data without impacting standard debug functionality.

5.5 Expected Outcomes

This Specification will standardize the data structure of configuration registers and status registers, and accessing method. It will help the customers of data center on rapid identification of system failures, precise locating of failing components, reducing time for root cause analysis (RCA), efficient hardware replacement and repair workflows, and support for pre-emptive maintenance based on failure risk prediction of the components. The results are expected to reduce system downtime and improve the overall efficiency of data center. This Specification will simplify the data center debugging.

6 Terminology

6.1 Syntax and Conventions

6.1.1 Special Terms

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL” in this Specification are to be interpreted as described in [BCP 14] [RFC2119] [RFC8174] when they appear.

6.1.2 Number Radix

This Specification may represent some numbers in non-decimal radix. In these cases, the following radix notations are used:

4’b0001	Binary representation of the decimal value 1 explicitly expressed in 4 bits.
0x5A	Hexadecimal representation of the decimal value 90.
7’h5A	Hexadecimal representation of the decimal value 90 explicitly expressed in 7 bits.

Note:

An underscore (i.e., ’_’) is sometimes used in the representation to improve readability. For example, an underscore can be used in a Hexadecimal number to separate each four digits in the example: 0x0001_2345.

6.1.3 JSON Keys

The JSON keys defined in this Specification will use Lower Camel Case when the key name is a concatenation of multiple words. Lower Camel Case is where the first letter of the first word is in lowercase and the first letters of subsequent words are in uppercase. For example, if the key name’s description is “number of nodes”, then the key’s name will be `numberOfNodes`.

In addition to this naming convention, key names written in this Specification will use the `Courier` font.

6.2 Definitions

- **Code Currency** - Describes how up-to-date, healthy, and maintainable a codebase is relative to current standards, tools, dependencies, and practices.
- **Hyperscalers** - Cloud service providers of the world, who value scaling and rapid time to market of new hardware products.
- **Life Cycle** -
- **Node** - A platform designed by a supplier integrating multiple components that run an operating system. These are sometimes also called compute nodes.

6.3 Abbreviations

DBG	Debug

6.4 Acronyms

BMC	Baseboard Management Controller
CPER	Common Platform Error Record
DVT	Design Validation Test
EVT	Engineering Validation Test
FW	Firmware
IMA	In-band Management Agent
JSON	JavaScript Object Notation
JTAG	Joint Test Action Group - Bare metal interface to access on-chip debug and test resources.
MCTP	Management Component Transport Protocol
MTTD	Mean Time to Debug
OCP	Open Compute Project
OEM	Original Equipment Manufacturer
OMA	Out-of-Band Management Agent
OOB	Out-of-Band
OS	Operating System
PLDM	Platform Level Data Model
PVT	Production Validation Test
RAS	Reliability, Availability, and Serviceability
RCA	Root Cause Analysis
SPDM	Security Protocol and Data Model
SW	Software
WDT	Watchdog Timer

7 Technical Overview (Informative)

7.1 Product Development Life Cycle

To better understand the requirements, usages, and restrictions on the various debug capabilities, it is important to understand the different environments and stages of progression through a product's development life cycle. Each stage of the Product Development Life Cycle represents a specific environment that has a certain set of expectations, behaviors, and limitations on what debug can be performed. These stages form a continuous lifecycle progression. As systems move through this progression, ownership transitions from silicon providers to OEMs and ultimately to datacenter operators and tenants, while debug access progressively transitions from unrestricted hardware-level visibility toward highly controlled management-based diagnostics and telemetry-driven operations. The Product Development Life Cycle is illustrated in the diagram below.

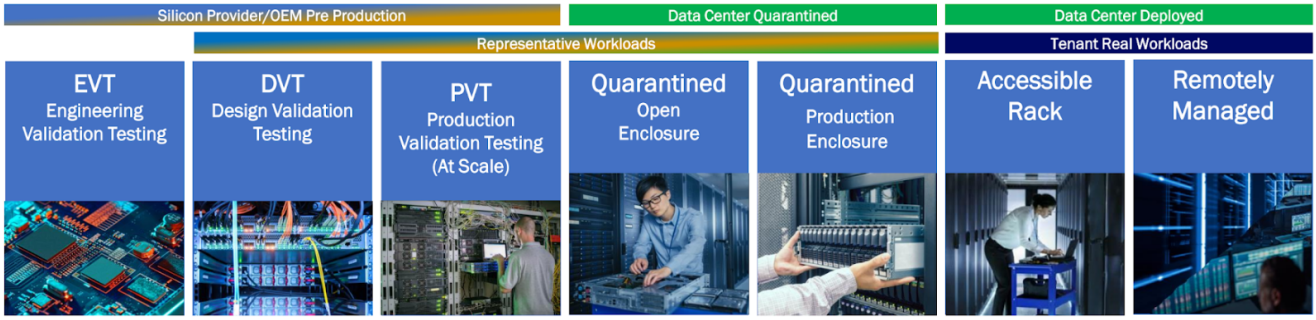

Figure 1: Product Development Life Cycle

This specification defines seven distinct debug environments that span the product lifecycle from silicon development through deployed datacenter operation. The environments differ in physical accessibility, ownership, security posture, and available debug capabilities. These stages are described in the following subsections.

It is important to note that the usual progression of the stages is from left to right in the diagram above. However, some systems may progress back from right to left due to issues that need to be resolved. For example, a system may be taken out of the production environments (e.g., Remotely Managed) to move left into the Quarantined or even the Production Validation Testing (PVT) environments to debug a complicated or large issue. Once an issue is solved, the system may move again toward the right, back to the production environments again.

7.1.1 Engineering Validation Test (EVT)

The Engineering Validation Test (EVT) stage represents the earliest platform validation phase within the silicon provider or OEM development lifecycle. During EVT, engineering teams validate initial hardware, firmware, and software functionality while developing platform bring-up procedures and identifying fundamental design issues. EVT is part of the Silicon Provider/OEM pre-production phase.

The platform is typically located in a development laboratory where physical access to the system is unrestricted. Engineers can directly access the platform, open the chassis, connect physical debug tools, modify firmware, and perform invasive debug operations. Physical debug interfaces such as JTAG are available and expected to be heavily utilized during this phase.

Primary objectives during this stage include:

- Silicon bring-up
- Hardware and firmware validation
- Feature verification
- Failure discovery and characterization
- Architecture validation
- Root-cause analysis of new issues

This environment supports deep platform visibility including hardware trace, error injection, memory access, system tracing, and privileged debug capabilities. Closed-loop debug and rapid design iteration are expected.

7.1.2 Design Validation Test (DVT)

The Design Validation Test (DVT) stage follows EVT and is used to validate that the platform design satisfies functional, reliability, performance, and interoperability requirements prior to production readiness. DVT remains within the Silicon Provider/OEM pre-production phase.

Systems continue to operate in laboratory settings with extensive physical access and broad debug capabilities available. While the platform is more mature than EVT, engineering teams still require access to low-level debug mechanisms to investigate defects discovered during validation testing.

Primary objectives during this stage include:

- Design verification
- Stress and reliability testing
- Performance characterization
- Failure reproduction
- Validation of corrective actions identified during EVT

Debug workflows focus on reproducing failures, collecting detailed traces, validating fixes, and ensuring production readiness. Physical debug interfaces continue to play a significant role in root-cause analysis activities.

7.1.3 Production Validation Testing (PVT)

Production Validation Testing (PVT) stage represents the final validation stage before large-scale deployment. It is used to validate manufacturing processes, production hardware, firmware releases, and operational readiness at scale.

Systems closely resemble production configurations and are representative of customer deployments. Platform behavior is evaluated using production software, production firmware, and realistic workloads while maintaining controlled validation conditions.

Primary objectives during this stage include:

- Production readiness validation
- Manufacturing verification
- Scale testing
- Final qualification
- Discovery of issues that only appear under large-scale deployment conditions

Debug capabilities become more restricted than EVT and DVT environments, with emphasis shifting toward production-compatible mechanisms, telemetry collection, management interfaces, and automated diagnostics.

7.1.4 Quarantined: Open Enclosure

The Quarantined: Open Enclosure stage is used when a deployed platform is removed from normal service and placed into an isolated environment for detailed diagnosis and failure analysis. The chassis is physically accessible and may be opened to allow direct access to components.

The system is quarantined from tenant workloads and production operations. Engineers have physical access to the hardware and can attach debug equipment while maintaining a configuration that is representative of the deployed platform.

Primary objectives during this stage include:

- Failure replication
- Signal tracing
- Live data collection
- In-depth system analysis
- Hardware failure characterization

This environment is specifically associated with failure replication, live trace capture, and in-depth silicon and system failure analysis. This environment acts as the bridge between production systems and laboratory-level investigative debugging.

7.1.5 Quarantined: Production Enclosure

The Quarantined: Production Enclosure stage is used to investigate production failures while preserving the deployed system configuration and physical enclosure. The system is removed from normal service but remains representative of its production state.

Although isolated from tenant workloads, the platform continues to operate in a production-like configuration. Physical intrusion is minimized to avoid altering failure conditions and to maintain environmental fidelity.

Primary objectives during this stage include:

- Failure discovery
- Failure characterization
- Failure triage
- Log collection
- Root-cause investigation

This environment is associated with collection and analysis of application, operating-system, BIOS, firmware, and hardware logs to discover, characterize, and triage new failures.

7.1.6 Production: Accessible Rack

The Production: Accessible Rack stage represents a deployed datacenter platform where physical access to the rack remains possible while the platform continues to operate in a datacenter setting. This environment is part of the deployed lifecycle and typically hosts representative or real workloads.

The rack remains physically accessible to datacenter personnel while maintaining production-like behavior. Debug operations are primarily performed through management interfaces though invasive physical interfaces can be used in this environment.

Primary objectives during this stage include:

- Diagnostics of deployed systems
- Maintenance operations
- Recovery activities
- Failure prevention
- Operational optimization

This environment supports collection of telemetry, diagnostics, and serviceability data while enabling maintenance workflows that require limited physical interaction with deployed infrastructure. It serves as an intermediate step between quarantined analysis and fully remote operation.

7.1.7 Production: - Remotely Managed

The Production: Remotely Managed stage represents the steady-state operational model for deployed datacenter systems. The platform executes tenant workloads while debug and management operations occur remotely only through standardized management interfaces.

Physical access is not assumed. Systems are managed through management networks, telemetry infrastructure, and out-of-band management mechanisms. The platform remains under strict production security controls while supporting diagnostics and recovery operations.

Primary objectives during this stage include:

- Platform monitoring
- Predictive failure analysis
- Automated diagnostics
- Maintenance
- Software and firmware updates
- Failure recovery
- Workload optimization

This environment is associated with remote maintenance, automated signature analysis, telemetry collection, trigger-based log capture, system updates, patch deployment, and recovery of known application, operating-system, and component failures as key activities in this environment. The environment also supports resident applications that optimize application, operating-system, and platform performance.

7.2 Execution Flow with Debug

It is critical that during the execution lifetime of a system, that the ability to perform progressively deeper data collection and root-cause analysis throughout the entire platform lifecycle, from boot through crash recovery, is available. A platform begins booting where firmware, hardware, and software initialization occur. Upon successful completion of boot activities, the platform transitions to the main execution state and enters normal operation. When executing, the platform is expected to deliver normal service and system functionality. If no issues are encountered, execution continues uninterrupted. However, as operational anomalies begin to emerge, the platform may enter different degraded or even halt/crash states. In degraded states, the platform remains functional but exhibits symptoms such as reduced throughput, increased latency, excessive resource consumption, or intermittent errors. If degradation worsens or corrective actions are not successful, the platform can progress to a halted/crashed (i.e., failure) state. In this condition, one or more platform functions are no longer operating correctly, and the escalation of failures may ultimately result in a platform becoming non-functional and normal execution

stops. At each stage, debug capabilities may be invoked to collect information, determine root cause, and attempt corrective action before the issue progresses to a more severe state.

The diagram below provides a generic execution flow that shows the different use cases where operational issues may occur. Note that boxes colored in green represent normal operating states, boxes in yellow represent degraded states, boxes in orange represent halted/crashed states, and boxes in blue represent debug states.

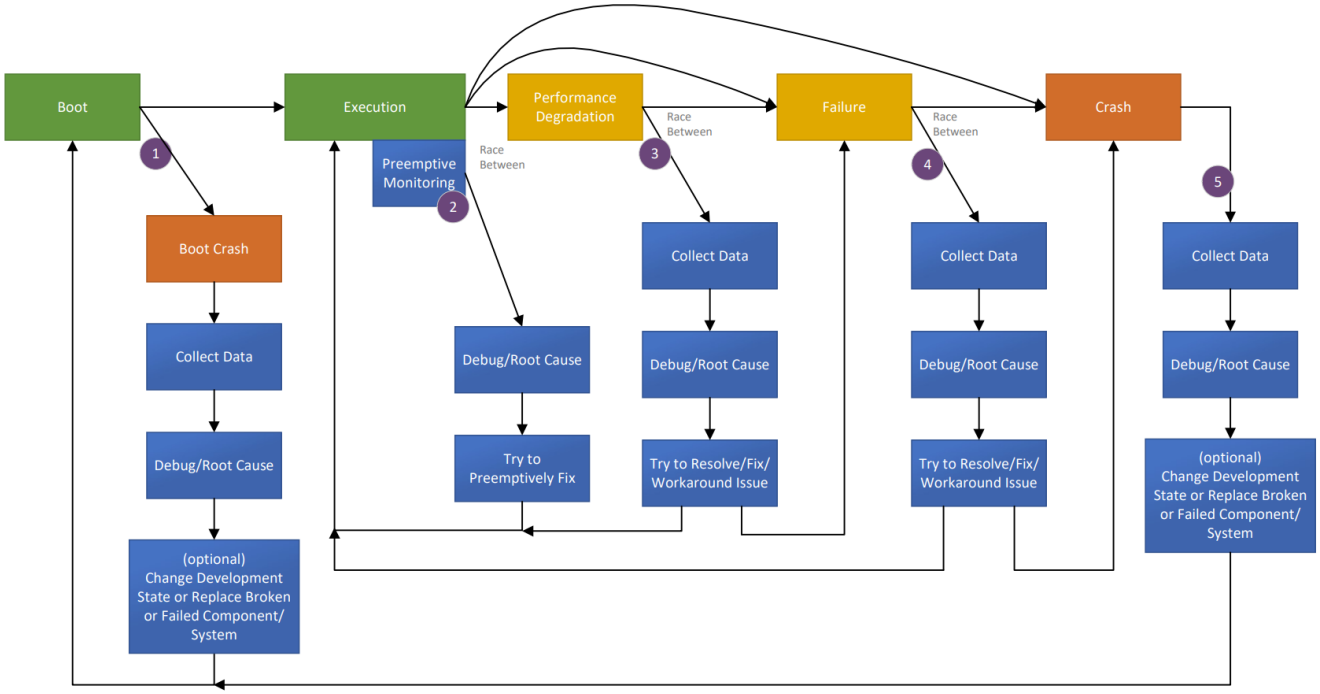

Figure 2: Execution Flow

To support this flow, the debug architecture needs to support the following:

- Pre-failure observability during normal execution.
- Data capture during degradation and failure conditions.
- Post-mortem crash analysis.
- Root-cause determination.
- Corrective action and validation.
- Return to normal operation while minimizing service disruption.

This lifecycle-oriented approach enables debug capabilities to improve platform availability, accelerate root-cause analysis, and reduce mean time to debug (MTTD) across all operational states. The main use cases, indicated by the numbered purple circles in the above diagram, are:

1. Boot Crash Handling
2. Preemptive Issues Handling
3. Performance Issues Handling
4. Failure Handling
5. Crash Handling

The next set of subsections describe these different use cases within this flow.

7.2.1 Boot Crash Handling

A boot crash occurs when the platform fails to successfully transition from the Boot state to the Execution state. The failure may occur during hardware initialization, firmware execution, security validation, configuration loading, training sequences, or other boot-time activities. This use case begins when boot execution starts and ends when the platform either enters the Execution state successfully or enters a terminal boot failure condition requiring remediation.

When a boot crash is detected:

1. The platform enters the Boot Crash state.
2. Debug capabilities are used to collect diagnostic data.
3. Root-cause analysis is performed. Verify whether the boot failure occurs consistently and determine dependency on platform configuration, software version, environmental conditions, or hardware population.
4. Corrective actions may be taken, including firmware modification, configuration updates, hardware repair or replacement, and/or component replacement.
5. The platform is rebooted and re-enters the Boot state.

It is important to note that boot-time visibility may be limited and the logging infrastructure may not yet be available. In addition, security policies may restrict debug access during early boot. The recovery mechanisms should preserve relevant failure context.

During this use case, the following are typical debug actions taken:

- Collect boot logs.
- Capture firmware trace information.
- Retrieve crash dumps or fault records.
- Analyze initialization sequence failures.
- Identify defective hardware or firmware components.

7.2.2 Preemptive Issues Handling

This use case occurs while the platform is operating normally in the Execution state. Monitoring and telemetry mechanisms detect indicators suggesting that a future failure may occur if corrective action is not taken. Rather than waiting for visible service degradation, debug operations are initiated proactively. This use case is triggered while the platform remains functional and ends when the condition is corrected or progresses into a degradation condition.

Examples of data collected to preempt issues include:

- Correctable error accumulation
- Thermal margin reduction
- Resource exhaustion trends
- Reliability threshold violations
- Predictive hardware health indicators

It is important to note that detection accuracy is critical. Excessive data collection should not significantly impact workload execution. In addition, actions need to minimize disruption to production operation.

Reproducibility can be difficult because failures have not yet fully manifested. Correlation across multiple telemetry samples may be necessary.

During this use case, the following are typical debug actions taken:

- Capture health telemetry.
- Collect performance counters.
- Analyze trends and predictive indicators.
- Apply preventative mitigations or configuration changes.

7.2.3 Performance Issues Handling

Performance degradation occurs when the platform continues to function but exhibits reduced operational effectiveness. This use case begins when measurable performance reduction occurs and ends when normal operation is restored or the issue progresses to failure. For this use case, there is a race between the issue progressing toward platform failure, and the debug infrastructure collecting sufficient information to diagnose the problem.

Possible causes of performance issues include:

- Resource contention
- Hardware degradation
- Firmware inefficiencies
- Excessive error recovery activity
- Thermal or power constraints

When performance is degraded, the following steps can be taken:

1. Debug capabilities are used to collect diagnostic data.
2. Root-cause analysis is performed. Determine whether the degradation is continuous, intermittent, workload dependent, and/or environment dependent.
3. Corrective actions may be taken, including firmware modification, configuration updates, hardware repair or replacement, and/or component replacement.

It is important to note that diagnostic activities should avoid further degrading performance. In this case, data collection may require prioritization due to resource constraints. Timely capture is important before symptoms disappear or worsen.

During this use case, the following are typical debug actions taken:

- Capture performance metrics.
- Collect traces and telemetry snapshots.
- Analyze resource utilization.
- Apply mitigations or corrective software and hardware changes.

7.2.4 Failure Handling

A failure state indicates that one or more platform functions are no longer operating correctly, although the system may still be partially operational. This use case begins when platform functionality is lost or significantly impaired and ends when either recovery occurs or the platform crashes. At this stage, there exists a race between the failure escalating leading to a crash and the debug capabilities capturing sufficient information for diagnosis and recovery.

Examples of different failures include:

- Device failures

- Link failures
- Memory subsystem failures
- Firmware service failures
- Platform service interruptions

When a failure occurs, the following steps can be taken:

1. Debug capabilities are used to collect diagnostic data.
2. Root-cause analysis is performed. Determine whether the failure is permanent, intermittent, or environment dependent.
3. Corrective actions may be taken, including firmware modification, configuration updates, hardware repair or replacement, and/or component replacement.

It is important to note that the time available for diagnostics may be limited. In some cases, some platform services may already be unavailable. Therefore, data preservation becomes increasingly important.

During this use case, the following are typical debug actions taken:

- Capture failure records.
- Collect dump data.
- Retrieve hardware status information.
- Analyze fault isolation information.
- Apply workaround, reset, repair, or replacement actions.

7.2.5 Crash Handling

A crash represents a terminal platform condition where normal execution has stopped. The platform is unable to continue operation and recovery typically requires reset, restart, repair, or component replacement. This use case begins when normal execution terminates and ends when corrective actions have been implemented and the platform can successfully reboot.

Following a crash:

1. Crash information is collected.
2. Root-cause analysis is performed. Determine whether the crash is deterministic or sporadic.
3. Corrective actions are taken, including firmware modification, configuration updates, hardware repair or replacement, and/or component replacement.
4. The platform is rebooted for validation and recovery. Validate that the corrective action eliminates the crash condition.

It is important to note that the diagnostic information may be lost if not captured immediately. Debug access may require specialized crash dump or post-mortem mechanisms. Also, security requirements must still be enforced during crash analysis.

During this use case, the following are typical debug actions taken:

- Capture crash dumps.
- Retrieve processor state.
- Collect memory contents.
- Capture hardware fault records.
- Perform post-mortem analysis.
- Identify required software, firmware, or hardware fixes.

7.3 High-Level Debug Architecture

The system debug architecture consists of in-band and out-of-band data collection paths, centralized aggregation, and analysis components.

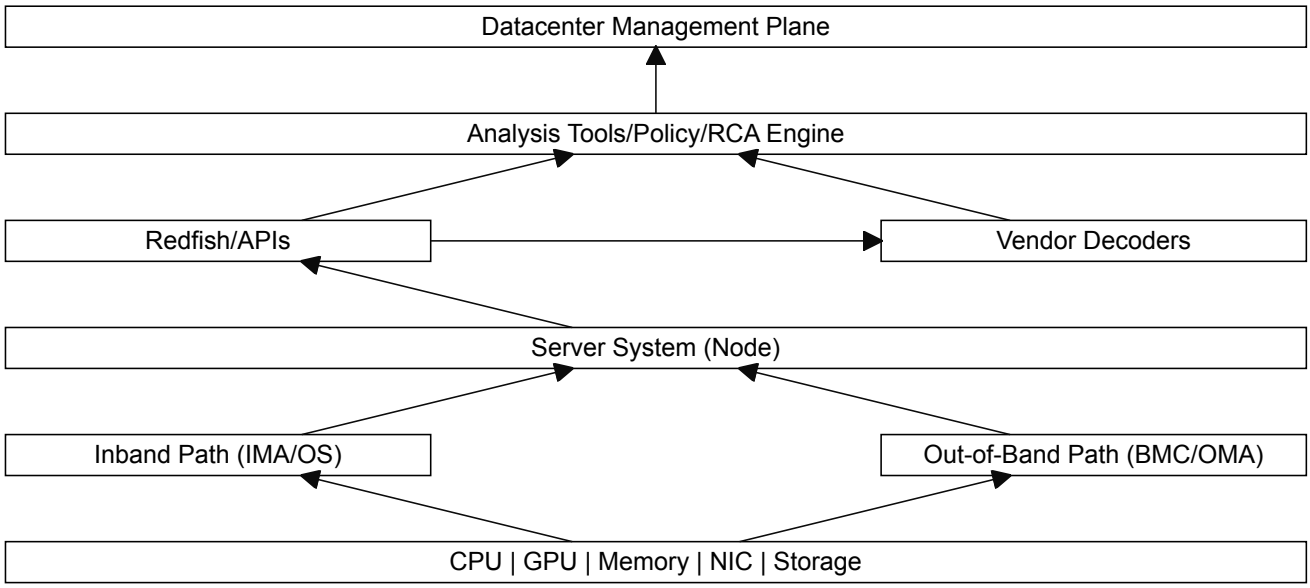

Figure 3: System Debug Architecture

Where:

- In-band path: Collects error data via OS, drivers, and kernel interfaces,
- IMA: In-band Management Agent,
- Out-of-band path: Collects error data independently of host software,
- OMA: Out-of-band Management Agent,
- Management plane: Performs analysis, correlation, and decision-making,
- Vendor decoders: Translate raw hardware data into actionable insights.

7.4 Configuration vs Physical Error Determination Flow

The system debug shall implement the following end-to-end debug data flow.

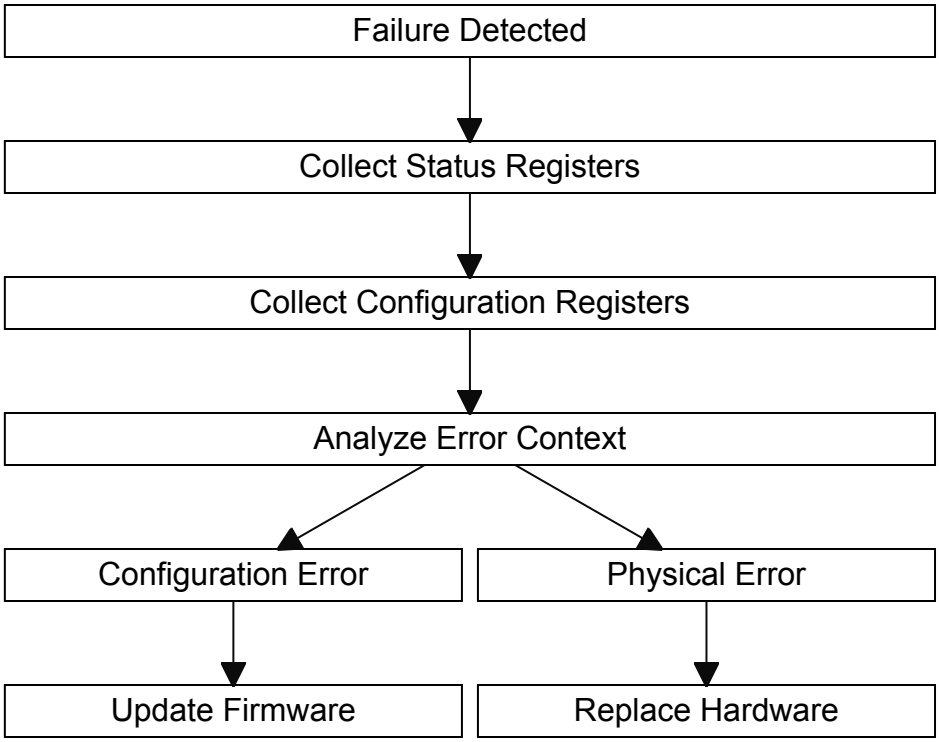

Figure 4: Configuration vs Physical Error Determination Flow

7.5 Content Arrangement

Configuration registers and status registers are primary mechanisms for device control and fault diagnosis. During system debugging, status registers shall be used to detect and localize failures. Configuration registers shall provide the settings for working operation. Failures shall be classified into the following categories: configuration errors, which caused by incorrect configuration, typically resolved through firmware or driver updates, and physical errors, which caused by hardware faults, requiring repair or replacement of components.

The system debug process shall include collecting status register data to identify failing components, collecting configuration register data to scan firmware issues if there is no hardware failure, determining failure classification (configuration vs. physical), and recommending of remediation actions.

The Baseboard Management Controller (BMC) shall act as the central entity for system-level debugging. The BMC shall collect and aggregate debug data across system components, identify failing devices based on status information, determine whether a failure is hardware-related or configuration-related, and provide diagnostic information and guidance for remediation.

In this Specification, we will define the data structure and the accessing methods for the necessary configuration registers and status registers of a given system. Standardized data structure and machine readable method to retrieve all relevant information about the configuration of a given system will be discussed in Code Currency chapters. The data format and dumping method of the status registers of a given system will be described in Status Dump chapters. The combination of configuration registers and status registers shall enable accurate fault identification and root cause analysis.

8 Layering

9 Handling Multiple Users

10 Security and Privacy

Debug capabilities are foundational to the development, deployment, and operation of modern datacenter platforms. Within the OCP ecosystem, debug is a critical enabler for system validation, fleet bring-up, fault isolation, reliability improvement, and root-cause analysis across heterogeneous, large-scale deployments. At the same time, debug interfaces represent one of the most powerful and sensitive control paths into a system. If not properly secured, they can undermine isolation boundaries, expose confidential and/or private tenant data, and compromise the integrity and trustworthiness of the platform. As such, security and privacy controls for debug are a core requirement for open, scalable, and trustworthy datacenter infrastructure.

Datacenter systems fundamentally differ from traditional development or enterprise environments. They are deployed at scale, operated remotely, and continuously managed through standardized management planes. Physical access to systems is limited, and debug operations are frequently performed in production, often in response to hardware faults, RAS events, or workload-impacting failures. In multi-tenant and cloud environments, debug will coexist with strong isolation guarantees between tenants, workloads, and security/privacy domains. These realities require that debug be treated not as a local or ad-hoc capability, but as a defined, policy-driven, and auditable service within the datacenter ecosystem.

Within OCP-aligned systems, debug functionality spans multiple layers and components, including CPUs, accelerators, memory subsystems, interconnects, firmware, and platform controllers. Debug data may include state information, traces, logs, crash dumps, register contents, or configuration data, and may be generated automatically or on demand. This data often flows through a complex path: from on-die debug logic, through platform firmware and runtime software, across sideband or management controllers (such as BMCs), and over standardized management or transport interfaces to remote debug and analytics tooling. These paths frequently traverse shared fabrics and standardized transports (e.g., PCIe, UCIe, USB, I3C, Ethernet), reflecting the composable and modular nature of modern datacenter designs.

The openness and standardization that characterize the OCP ecosystem amplify both the value and the risk of debug capabilities. Open interfaces, interoperable tooling, and vendor-agnostic management frameworks enable broad ecosystem participation and innovation, but they also require clearly defined security boundaries and consistent enforcement across vendors, components, and system architectures. Debug security and privacy mechanisms therefore need to be composable, transport-agnostic, and interoperable, while still providing strong guarantees of authentication, authorization, confidentiality, and integrity.

Debug data is inherently high-value. It may expose firmware state, internal hardware behavior, implementation details of vendor IP, workload-specific data, or security-sensitive information such as keys, identifiers, or policy state. In the datacenter, uncontrolled disclosure or manipulation of such data can have fleet-wide impact, affecting availability, data confidentiality, compliance, and customer trust. As a result, debug operations must be explicitly authorized, tightly scoped, and continuously observable, with protections applied end-to-end, from the debug initiator to the target component and back to the consuming tools.

Securing debug in an open datacenter environment also requires careful consideration of lifecycle state and operational context. Systems transition through development, manufacturing, provisioning, deployment, and end-of-life phases, each with different debug requirements and risk profiles (as described in). Likewise, production debug workflows may differ significantly from lab workflows, favoring non-intrusive data collection, automation, and policy-based enablement over manual, invasive

control. A robust security-for-debug architecture must accommodate these variations without relying on proprietary assumptions or physical trust.

The goal is to establish principles and requirements to ensure that debug capabilities remain powerful enough to support large-scale operations while preserving platform security, protecting tenant and workload data, and maintaining trust across an open, multi-vendor ecosystem. Debug need to be first-class, secure, and an interoperable capability that scales with the needs of modern datacenters.

10.1 Debug Security Threat Model

This section describes the threat model for debug capabilities in open datacenter systems and establishes the security assumptions, assets, adversaries, and threat classes that must be considered when defining secure debug architectures. The intent is not to prescribe a specific implementation, but to provide a common, ecosystem-aligned framework to guide requirements, design decisions, and interoperability.

10.1.1 Assets and Security Objectives

Debug capabilities expose high-value assets that require explicit protection in datacenter environments. These assets include, but are not limited to:

- **Debug Data** - Register state, memory contents, trace streams, crash dumps, logs, configuration data, and firmware or silicon state.
- **Security-Sensitive Material** - Cryptographic keys, device identifiers, security configuration, lifecycle state, entitlement or policy state, and vendor- or tenant-specific IP.
- **Control Capabilities** - Invasive debug operations such as run control, memory modification, register writes, and fault injection mechanisms.
- **Platform Trust Guarantees** - Isolation boundaries between tenants, security domains, execution worlds, and privilege levels.

The primary security objectives for debug in the datacenter are:

- **Confidentiality** - Prevent unauthorized disclosure of debug data.
- **Integrity** - Prevent unauthorized modification of debug data or system state.
- **Authenticity and Authorization** - Ensure that all debug actions are initiated by authenticated and explicitly authorized entities.
- **Availability** - Ensure debug does not undermine system availability, fleet stability, or workload isolation.
- **Auditability** - Ensure debug operations are observable, attributable, and traceable for operational and compliance purposes.

10.1.2 Trust Boundaries and Assumptions

Datacenter systems are composed of multiple trust domains, often owned or managed by different parties. The debug threat model assumes the following:

- **No Physical Trust Assumption** - Physical access alone does not imply authorization for debug.
- **Remote Operation by Default** - Debug is initiated and consumed remotely via management and datacenter fabrics.
- **Shared Infrastructure** - Debug traffic may traverse shared transports, controllers, and fabrics not dedicated exclusively to debug.

- **Lifecycle Awareness** - Systems transition through multiple lifecycle states, each with different debug expectations and risk profiles.

Trust boundaries typically exist between:

- Silicon components and platform firmware
- Platform firmware and host software
- Host software and management controllers (e.g., BMC)
- Management controllers and external debug or fleet tooling
- Different tenants, workloads, or execution domains within the same system

A secure debug architecture explicitly needs to define how debug requests and data cross these boundaries and what policies govern each transition.

10.1.3 Threat Actors

The following threat actor classes are considered within scope:

- **External Network Attackers** - Attempting to intercept, inject, or replay debug traffic over management or data networks.
- **Insider Threats** - Authorized personnel or tooling misusing debug capabilities beyond intended scope.
- **Compromised Components** - A compromised BMC, host OS, or management service attempting to escalate privileges via debug paths.
- **Supply Chain or Lifecycle Abuse** - Exploitation of debug capabilities unintentionally left enabled or misconfigured during manufacturing, provisioning, and/or decommissioning.
- **Cross-Tenant Adversaries** - Attempts to access or infer another tenant's data or state through debug mechanisms.

10.1.4 Threat Classes

The following threat classes are representative and non-exhaustive:

- **Unauthorized Debug Enablement** - Attackers may attempt to enable or retain debug capabilities in production systems without proper authorization, bypassing intended lifecycle or policy controls.
- **Debug Data Exfiltration** - Debug interfaces may be leveraged to extract sensitive data, including memory contents, firmware state, or security material, either directly or indirectly via side channels.
- **Privilege Escalation via Debug** - Invasive debug operations may allow attackers to bypass software or hardware enforcement mechanisms, crossing execution worlds or privilege domains.
- **Tampering and Injection** - Attackers may inject malicious debug commands or alter debug data in transit, leading to corrupted diagnostics, misattribution, or system compromise.
- **Replay and Spoofing** - Previously authorized debug sessions or messages may be replayed or spoofed to gain unauthorized access or influence system behavior.
- **Platform-Wide Impact** - Because debug is often highly privileged, its abuse can have fleet-wide consequences, impacting availability, security posture, or compliance across many systems.

10.1.5 Threat Model Implications

Given these threats, debug architectures need to adhere to the following principles:

- **Explicit Enablement** - Debug capabilities default to disabled and require deliberate, authenticated, and policy-controlled enablement.

- **End-to-End Protection** - Debug requests and data need to be protected across all hops, regardless of transport or intermediate components.
- **Least Privilege and Scoping** - Debug access needs to be scoped to specific components, functions, time windows, and data sets.
- **Transport and Implementation Independence** - Security properties do not rely on a single transport, vendor-specific mechanism, or physical assumption.
- **Observability and Accountability** - Debug operations are logged and attributable to support operational visibility and forensic analysis.

10.2 Debug Security Architecture

This section describes a reference architectural model for securing debug in open datacenter platforms. The model is descriptive rather than prescriptive, enabling multiple implementations that meet the same security objectives.

10.2.1 Architectural Layers

A debug security architecture typically spans the following layers:

- **Target Components** - CPUs, accelerators, memory subsystems, interconnects, and other silicon blocks that generate or consume debug data.
- **On-Device Enforcement** - Hardware and firmware mechanisms that gate debug access, enforce policies, and protect data.
- **Platform Management** - Management controllers (e.g., BMCs) acting as aggregation points for debug control and data flow.
- **Management and Data Fabrics** - Standardized transports and protocols used to carry debug traffic alongside other management traffic.
- **Remote Debug and Fleet Tooling** - Authorized tools used by operators, automation systems, or analytics platforms.

Security controls need to be applied coherently across all layers, with explicit trust boundary definitions between them.

10.2.2 Policy Flow and Enforcement

The debug policy originates from an authorized management authority within the datacenter control plane and is propagated through the platform to the components that participate in debug enablement and data handling. Policy information is represented in a form suitable for automated processing and distribution across heterogeneous system components.

The policy's scope is expressed declaratively, identifying the allowed debug capabilities, affected components, visibility constraints, and applicable operational context. As debug requests traverse the system, each participating layer evaluates the policy material relevant to its role and applies local enforcement consistent with its trust boundary. This distributed evaluation model avoids reliance on a single enforcement point (i.e., defense in depth) and supports scalability across large fleets.

Intermediate components act as conduits for the debug policy and debug traffic rather than sources of additional authority. Policy interpretation at these layers preserves the original scope and intent defined by the policy originator. Where multiple enforcement layers are present, debug access reflects the intersection of allowed capabilities rather than an accumulation of privileges.

Policy updates, revocation, and expiration are treated as first-class operations. Changes in policy state propagate through the same channels used for initial enablement, enabling timely reduction or removal of debug access in response to lifecycle transitions, operational events, or security considerations.

10.2.3 Interoperability Considerations

Debug security mechanisms operate within a multi-vendor, open ecosystem and therefore benefit from alignment with broadly adopted datacenter management and security frameworks. Common abstractions for authentication, authorization, secure session establishment, and policy expression enable integration across components sourced from different vendors.

Interoperability is supported by separating security intent from transport specifics. Debug security properties remain consistent whether debug traffic flows over sideband interfaces, in-band fabrics, or network-based transports. This separation allows platforms to evolve or compose different physical and logical topologies without redefining debug security semantics.

Vendor-specific extensions appear as additive capabilities layered on top of the common security framework. Such extensions coexist with baseline mechanisms and preserve consistent behavior when interoperating with ecosystem tooling. Debug operations across mixed deployments reflect the shared security model rather than vendor-unique assumptions.

From an operational perspective, interoperability also encompasses tooling and observability. Debug security state, access events, and audit information integrate naturally into platform management systems, enabling operators to reason about debug behavior across platforms using common workflows and representations.

10.3 Communication Protection

In contemporary datacenter environments, debug communication frequently extends beyond local physical access. Debug tools connect remotely through management controllers, sideband fabrics, or network-based transports, often traversing multiple intermediaries before reaching the target device. In these environments, debug traffic often traverses components that are not intrinsic to the debug function itself, including baseboard management controllers, network adapters, protocol bridges, and shared fabrics. Therefore, the transport itself becomes part of the system's security boundary.

Protecting debug communication therefore encompasses more than controlling whether debug is enabled. It also addresses how debug data is conveyed between a debug initiator and the target. Without explicit protection, debug traffic is susceptible to interception, modification, replay, or unintended disclosure as it transits internal buses, board-level interconnects, or external networks. These risks increase in at-scale deployments where multi-tenant environments, remote access, and open ecosystem integration are common.

A secured debug transport establishes confidence that debug traffic reaches only its intended destination, originates from an authenticated source, and remains intact throughout its path. This protection applies consistently across different physical interfaces and transport technologies, including traditional local debug ports, sideband management channels, and packet-based network transports. The security properties of the debug session remain independent of the underlying medium, allowing platforms to evolve connectivity options without redefining debug trust assumptions.

Equally important, debug transport protection aligns with the principle of least exposure. Only debug data explicitly authorized by platform policy becomes observable over the transport, and visibility remains constrained to the scope associated with the active debug context. By binding debug data flow

to authenticated identity, approved capability sets, and defined lifecycle states, transport-level protection complements on-target debug gating mechanisms.

10.3.1 End-to-End Trust Assumptions

Protecting debug communication relies on a clear understanding of the trust relationships between the entities involved in a debug session and the infrastructure that carries debug traffic. An end-to-end trust model establishes which elements participate actively in security decisions, which elements are treated as untrusted or opaque, and how confidence in the debug exchange is derived.

10.3.1.1 Trust Anchors and Endpoint Identity

The primary trust anchors in a debug communication flow reside at the endpoints of the debug session: the debug initiator and the target system. Each endpoint maintains its own cryptographic identity and security state, forming the basis for authentication and secure session establishment. Trust in debug communication derives from the ability of these endpoints to authenticate one another and to associate the resulting session with a known device identity and security posture.

Identity verification occurs directly between the communicating endpoints rather than being delegated to intermediate components. Device identity, attestation state, and cryptographic capabilities are exchanged as part of session establishment, allowing both sides to reason about the authenticity of the peer before sensitive debug data is exchanged.

10.3.1.2 Role of Intermediate Components

Between the debug initiator and target system, debug traffic can traverse one or more intermediate elements such as management controllers, protocol bridges, switches, or network infrastructure. In the end-to-end trust model, these intermediaries are treated as transport participants rather than trust decision-makers.

Intermediate components forward debug traffic without possessing the cryptographic keys associated with the debug session. Visibility into debug payloads is not assumed, and the security of the debug exchange does not depend on the correctness, confidentiality, or privilege level of these components. Secure debug sessions span across interface bridges and shared transports without expanding the trusted computing base.

10.3.1.3 Secure Session Scope and Boundaries

A debug session represents a bounded trust relationship defined by authenticated endpoints, negotiated cryptographic parameters, and an agreed session lifetime. Within this boundary, debug data exchanges benefit from confidentiality and integrity protection that remains consistent regardless of the path taken through the system.

The existence of a secure session does not, by itself, imply unrestricted debug access. Instead, it provides a protected channel within which debug semantics are evaluated according to platform policy and lifecycle state. Debug operations occur within the intersection of session trust and authorization scope, preserving separation between communication security and functional enablement.

10.3.1.4 Separation of Trust and Authorization

End-to-end trust assumptions distinguish between who is communicating and what that entity is permitted to do. Session establishment builds confidence in the identity and integrity of the

communicating endpoints, while authorization decisions determine the visibility and control exposed through the debug interface.

This separation allows the same debug transport security model to apply across multiple operational contexts, such as development, manufacturing, or controlled field diagnostics. Changes in lifecycle state or debug policy influence authorization outcomes without altering the underlying trust relationship formed during session establishment.

10.3.1.5 Resilience to Partial Compromise

The end-to-end trust model assumes that some elements along the debug communication path can be compromised, misconfigured, or shared with other tenants. By limiting trust to authenticated endpoints and protecting debug payloads cryptographically, the model confines the impact of such compromise.

A compromised intermediary can observe traffic patterns, delay or drop messages, or disrupt availability, but it lacks the ability to inspect or modify debug payloads without detection. Similarly, replay or injection attempts outside the established session context fail to produce meaningful debug effects at the target endpoint.

10.3.1.6 Alignment with Open Datacenter Deployments

In open datacenter environments, platforms frequently integrate components from multiple vendors and operate under shared management infrastructure. The end-to-end trust assumptions described here enable interoperability by avoiding implicit trust in platform composition or topology.

Debug communication remains anchored to endpoint identity and session context rather than vendor-specific wiring or isolation guarantees. This approach supports scalable, remote debug workflows while preserving consistent security behavior across heterogeneous platforms and evolving interconnect technologies.

10.4 Requirements

10.4.1 Debug Enablement and Lifecycle Control

- **[SEC-0001]** Debug capabilities shall be disabled by default in production systems.
- **[SEC-0002]** Debug enablement shall be explicitly authorized and bound to a well-defined system lifecycle state (e.g., development, provisioning, deployed, decommissioned).
- **[SEC-0003]** The mechanism used to enable debug shall not rely on physical access alone as an authorization signal.
- **[SEC-0004]** Debug enablement shall be reversible and support explicit disablement or expiration, including automatic revocation after a defined time window or event.

10.4.2 Authentication and Authorization

- **[SEC-0005]** All debug access shall require strong authentication of the requesting entity.
- **[SEC-0006]** Authorization of debug requests shall be policy-driven, allowing debug operations to be scoped by:
  - Component or functional block
  - Debug capability class (e.g., non-invasive vs. invasive)
  - Data type or visibility level
  - Time and operational context

- **[SEC-0007]** Debug authorization decisions shall be enforceable at the lowest practical layer, including within silicon where feasible.

10.4.3 Debug Data Protection

- **[SEC-0008]** Debug data shall be protected for confidentiality and integrity end-to-end, from the point of generation to final consumption by authorized tools.
- **[SEC-0009]** Cryptographic protection shall not assume trust in intermediate transports, bridges, and/or controllers.
- **[SEC-0010]** Debug data stored at rest (e.g., crash dumps, logs, trace buffers) shall be protected against unauthorized access and modification.
- **[SEC-0011]** Implementations should support data minimization and filtering to limit exposure of tenant or workload-specific data.

10.4.4 Isolation and Containment

- **[SEC-0012]** Debug operations shall not bypass or weaken isolation guarantees between:
  - Tenants
  - Security domains
  - Privilege levels
  - Execution worlds or contexts
- **[SEC-0013]** Invasive debug capabilities shall be explicitly distinguishable from non-invasive debug and subject to stricter authorization and policy controls.
- **[SEC-0014]** Debug access to one component shall not implicitly grant access to other components or system functions.

10.4.5 Transport and Interface Independence

- **[SEC-0015]** Debug security properties shall be preserved regardless of transport, including but not limited to PCIe, UCIe, USB, I3C, Ethernet, or sideband interfaces.
- **[SEC-0016]** Security shall not depend on a single proprietary protocol or debug interface.
- **[SEC-0017]** Debug capabilities/mechanisms/communication should support multiplexing over shared datacenter fabrics without exposing debug traffic to unauthorized entities.

10.4.6 Observability and Auditability

- **[SEC-0018]** Debug operations shall be observable and auditable through the platform management stack.
- **[SEC-0019]** Audit records shall include:
  - Identity of the requesting entity
  - Authorized debug scope
  - Time and duration of access
  - Components and data accessed
- **[SEC-0020]** Audit data shall be protected against tampering and retained in accordance with datacenter operational policies.

11 Data Stacks

11.1 Code Currency

Code Currency is a term used to describe how up-to-date, healthy, and maintainable a codebase is relative to current standards, tools, dependencies, and practices. This codebase includes, but not limited to, the following:

- Programming languages and compiler versions
- Libraries, frameworks, and SDKs
- Toolchains, build systems, and CI/CD infrastructure
- Security patches and vulnerability fixes

Code Currency refers to the timeliness, consistency, and lineage awareness of code and configuration artifacts that participate in debug enablement, control, and analysis. In the context of debug architectures, maintaining accurate code currency is essential to ensure that debug capabilities operate predictably, securely, and in alignment with platform policy across the system lifecycle.

Debug mechanisms inherently bridge development, manufacturing, validation, and field operation. As a result, debug flows often depend on a diverse set of artifacts—including firmware binaries, microcode, configuration tables, entitlement credentials, policy definitions, and tooling metadata—that evolve independently over time. Without explicit consideration of code currency, mismatches between these artifacts can introduce ambiguity in debug behavior, weaken security guarantees, and complicate auditability and incident response.

More often than not, the root cause of a failure or problem is simply that the system has not been configured correctly and/or it is running either outdated firmware, an incompatible combination of software/firmware components, or components that have not been configured appropriately. The concept of Code Currency in this Specification is used to address this issue by providing a one-stop shop style standardized machine readable method to retrieve all relevant information about the configuration of a given system. By explicitly accounting for these forms of data covered by Code Currency, the debug architecture can reason about compatibility, enforce policy consistently, and detect invalid or unsafe combinations of code and authorization. Treating Code Currency as an architectural concern—rather than an implicit assumption—helps ensure that debug capabilities remain controlled, auditable, and resilient as the platform evolves over time.

11.1.1 File Format and Schema

The information for Code Currency will be exposed by a system as a file in the Redfish profile of the system. Components and subsystem will expose their part of the information to the Main BMC as PLDM type 7 files. The content of the file is organized using the JSON schema defined by Requirement [CC-0001]. The relationship diagram below outlines this JSON schema.

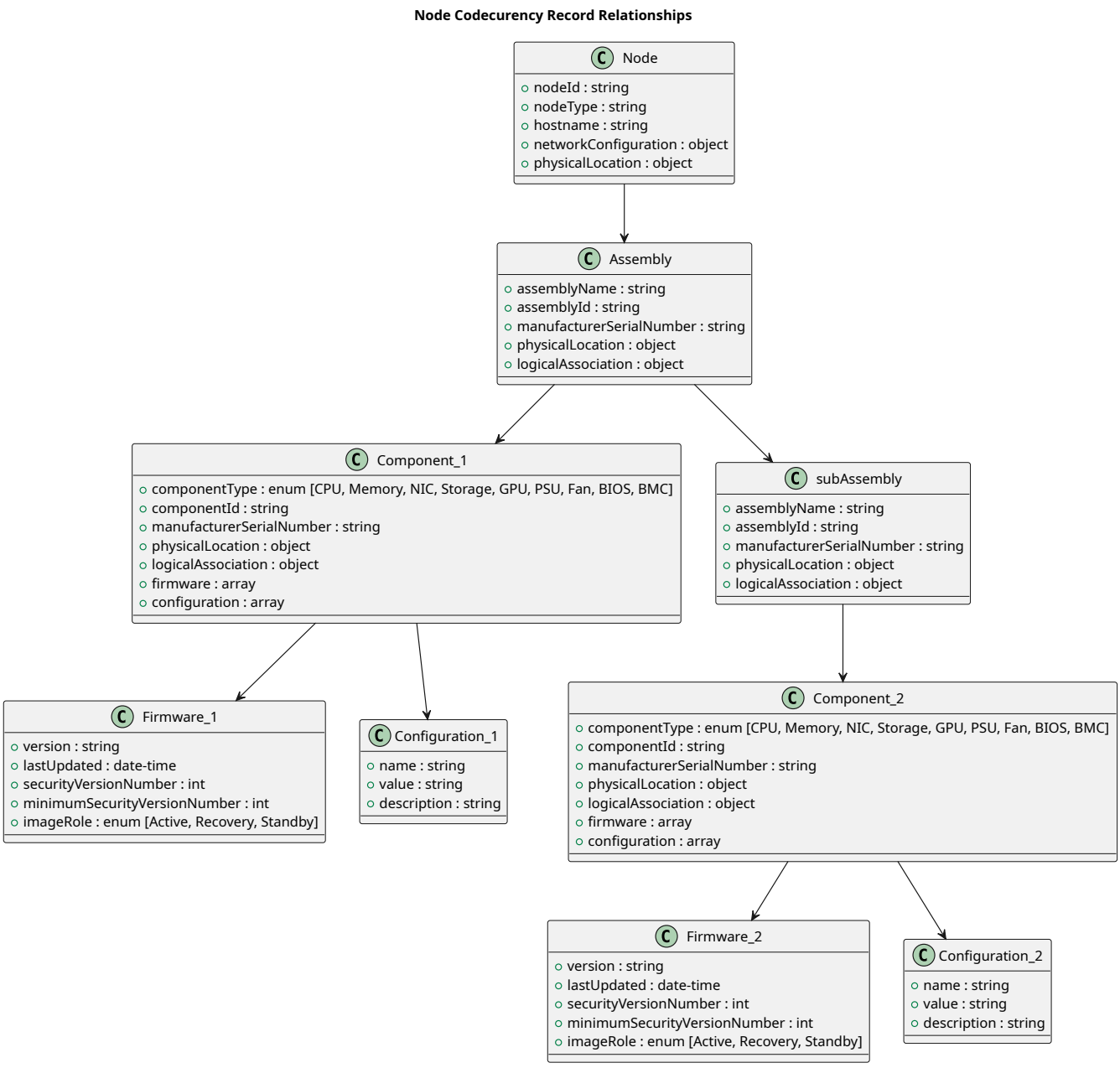

Figure 5: Code Currency Relationships

The table below defines the elements of the node-level for the Code Currency JSON schema.

Table 1: Code Currency Node-Level Elements Definition

Element Name	Description
`hostname`	Hostname for network identification.

(continued on next page)

(continued from previous page)

Element Name	Description
networkConfiguration.ipAddresses	List of IPv4 addresses. (required)
networkConfiguration.macAddresses	MAC addresses for network interfaces.
networkConfiguration.ports	Open or configured ports.
nodeId	Unique identifier for the node. (required)
nodeType	Type of node (e.g., AI Server, Ethernet Switch). (required)
physicalLocation.datacenter	Physical placement identifier.
physicalLocation.rack	Physical placement identifier.
physicalLocation.row	Physical placement identifier.
physicalLocation.placement.bayId	Bay-specific identifier if type=Bay.
physicalLocation.placement.bayPosition	Bay-specific position if type=Bay.
physicalLocation.placement.heightInU	Height in rack units if type=RackSlot.
physicalLocation.placement.slot	Rack unit position if type=RackSlot.
physicalLocation.placement.type	RackSlot or Bay. (required)

The table below defines the elements of the “assembly” element for the Code Currency JSON schema.

Table 2: Code Currency “assembly” Elements Definition

Element Name	Description
assemblyName	Name of the assembly (e.g., Baseboard). (required)
assemblyId	Unique identifier. (required)
components	Array of components.
logicalAssociation	Array of logical connectivity details for this assembly.
logicalAssociation.links.connectedTo	What the link is connected to. (required)
logicalAssociation.links.laneCount	Number of lanes for the link.
logicalAssociation.links.protocol	The protocol of the link. (required)
manufacturerSerialNumber	Manufacturer’s serial number for the assembly.
physicalLocation.slot	Physical slot identifier where this assembly is installed.
physicalLocation.position	Position within the node or assembly where this assembly is installed.
subAssemblies	Nested assemblies.

The table below defines the elements of the “components” element for the Code Currency JSON schema.

Table 3: Code Currency “components” Elements Definition

Element Name	Description
componentId	Unique identifier. (required)
componentType	Type of component. Permitted values: CPU, Memory, NIC, Storage, GPU, PSU, Fan, BIOS, BMC, AMC. (required)
configuration	Array of custom configuration attributes for the component.
configuration.description	Description of the attribute.
configuration.name	Configuration attribute name. (required)
configuration.value	Configuration attribute value. (required)
firmware	Array of firmware entries for the component. (required)
firmware.imageRole	Indicates if this firmware is actively running or a redundant recovery image. Permitted values: Active, Recovery, or Standby.
firmware.lastUpdated	Timestamp of last update.
firmware.minimumSecurityVersionNumber	Minimum enforced SVN.
firmware.securityVersionNumber	Current Security Version Number (SVN).
firmware.version	Firmware version (required).
logicalAssociation	Array of logical connectivity details for this component.
logicalAssociation.links.connectedTo	What the link is connected to. (required)
logicalAssociation.links.laneCount	Number of lanes for the link.
logicalAssociation.links.protocol	The protocol of the link. (required)
manufacturerSerialNumber	Manufacturer’s serial number.
physicalLocation.position	Position within the node or assembly where this component is installed.
physicalLocation.slot	Physical slot identifier where this component is installed.

Examples of the JSON files following the given schema can be found in Section A.

11.1.2 Code Currency Redfish Schema

11.1.3 Requirements

- **[CC-0001]** The Code Currency log shall be in JSON format with the following schema:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Node Code Currency Record",
  "type": "object",
  "properties": {
    "nodeId": { "type": "string", "description": "Unique identifier for the node" },
    "nodeType": { "type": "string", "description": "Type of node (e.g., AI Server, Ethernet Switch,
    ↪  Management Tray)" },
    "hostname": { "type": "string", "description": "Node hostname" },
    "networkConfiguration": {
```

```
      "type": "object",
      "description": "Network configuration details for the node",
      "properties": {
        "ipAddresses": { "type": "array", "items": { "type": "string", "format": "ipv4" } },
        "ports": { "type": "array", "items": { "type": "integer", "minimum": 1, "maximum": 65535 }
        ↪  },
        "macAddresses": { "type": "array", "items": { "type": "string", "pattern":
        ↪  "^([0-9A-Fa-f]{2}:){5}[0-9A-Fa-f]{2}$" } }
      },
      "required": ["ipAddresses"]
    },
    "physicalLocation": {
      "type": "object",
      "description": "Physical location of the node in the datacenter",
      "properties": {
        "rack": { "type": "string" },
        "row": { "type": "string" },
        "datacenter": { "type": "string" },
        "placement": {
          "type": "object",
          "properties": {
            "type": { "type": "string", "enum": ["RackSlot", "Bay"] },
            "slot": { "type": "integer", "description": "Rack unit position if RackSlot" },
            "heightInU": { "type": "integer", "description": "Height in U if RackSlot" },
            "bayId": { "type": "string", "description": "Bay identifier if Bay" },
            "bayPosition": { "type": "string", "description": "Position within the bay" }
          },
          "required": ["type"]
        }
      },
      "required": ["rack", "row", "datacenter", "placement"]
    },
    "assemblies": {
      "type": "array",
      "description": "List of assemblies within the node (supports nested assemblies)",
      "items": { "$ref": "#/$defs/assembly" }
    }
  },
  "required": ["nodeId", "nodeType", "assemblies"],
  "$defs": {
    "assembly": {
      "type": "object",
      "properties": {
        "assemblyName": { "type": "string" },
        "assemblyId": { "type": "string" },
        "manufacturerSerialNumber": { "type": "string", "description": "Manufacturer's serial
        ↪  number for the assembly" },
        "physicalLocation": {
          "type": "object",
          "description": "Physical slot or bay where this assembly is installed",
          "properties": {
            "slot": { "type": "string", "description": "Physical slot identifier (e.g., PCIe slot,
            ↪  OCP slot)" },
            "position": { "type": "string", "description": "Position within the node or assembly" }
          }
        },
        "logicalAssociation": {
          "type": "object",
          "description": "Logical connectivity details for this assembly",
          "properties": {
            "links": {
              "type": "array",
              "description": "List of logical links consumed by this assembly",
              "items": {
                "type": "object",
                "properties": {
                  "connectedTo": { "type": "string" },
                  "protocol": { "type": "string" },
```

```
              "laneCount": { "type": "integer" }
            },
            "required": ["connectedTo", "protocol"]
          }
        }
      }
    },
    "components": { "type": "array", "items": { "$ref": "#/$defs/component" } },
    "subAssemblies": { "type": "array", "items": { "$ref": "#/$defs/assembly" } }
  },
  "required": ["assemblyName", "assemblyId"]
},
"component": {
  "type": "object",
  "properties": {
    "componentType": { "type": "string", "enum": ["CPU", "Memory", "NIC", "Storage", "GPU",
    ↪  "PSU", "Fan", "BIOS", "BMC", "AMC"] },
    "componentId": { "type": "string" },
    "manufacturerSerialNumber": { "type": "string", "description": "Manufacturer's serial
    ↪  number for the component" },
    "physicalLocation": {
      "type": "object",
      "description": "Physical slot or bay where this component is installed",
      "properties": {
        "slot": { "type": "string", "description": "Physical slot identifier (e.g., PCIe slot,
        ↪  DIMM slot)" },
        "position": { "type": "string", "description": "Position within the node or assembly" }
      }
    },
    "logicalAssociation": {
      "type": "object",
      "description": "Logical connectivity details for this component",
      "properties": {
        "inks": {
          "type": "array",
          "description": "List of logical links consumed by this component",
          "items": {
            "type": "object",
            "properties": {
              "connectedTo": { "type": "string" },
              "protocol": { "type": "string" },
              "laneCount": { "type": "integer" }
            },
            "required": ["connectedTo", "protocol"]
          }
        }
      }
    },
    "firmware": {
      "type": "array",
      "description": "List of firmware entries for the component",
      "items": {
        "type": "object",
        "properties": {
          "version": { "type": "string", "description": "Firmware version" },
          "lastUpdated": { "type": "string", "format": "date-time", "description": "Timestamp
          ↪  of last firmware update" },
          "securityVersionNumber": { "type": "integer", "description": "Current Security
          ↪  Version Number (SVN)" },
          "minimumSecurityVersionNumber": { "type": "integer", "description": "Minimum enforced
          ↪  Security Version Number (SVN)" },
          "imageRole": {
            "type": "string",
            "enum": ["Active", "Recovery", "Standby"],
            "description": "Indicates if this firmware is actively running or a redundant
            ↪  recovery image"
          }
        },
```

```
            "required": ["version"]
          }
        },
        "configuration": {
          "type": "array",
          "description": "Custom configuration attributes for the component",
          "items": {
            "type": "object",
            "properties": {
              "name": { "type": "string", "description": "Configuration attribute name" },
              "value": { "type": "string", "description": "Configuration attribute value" },
              "description": { "type": "string", "description": "Optional description of the
                ↪  attribute" }
            },
            "required": ["name", "value"]
          }
        }
      },
      "required": ["componentType", "componentId", "firmware"]
    }
  }
}
```

11.2 Status Dump

11.3 Console

11.4 Interactive Debug/Run Control

12 Future Work

Appendix A: Code Currency Examples

Below is a JSON example for Code Currency of a hypothetical node with:

- Dual-socket CPUs
- Memory modules
- BMC
- Two NVMe drives
- Two NICs on a hot-swap backplane
- UBB subassembly with BMC
- Two OAM subassemblies (each with GPU and AMC)


```
{
  "nodeId": "AI-Node-001",
  "nodeType": "AI Server",
  "hostname": "ai-node-001.dc.local",
  "networkConfiguration": {
    "ipAddresses": ["192.168.10.101"],
    "ports": [22, 443],
    "macAddresses": ["00:1A:2B:3C:4D:5E"]
  },
  "physicalLocation": {
    "rack": "R42",
    "row": "Row-A",
    "datacenter": "DC-West",
    "placement": {
      "type": "RackSlot",
      "slot": 12,
      "heightInU": 2
    }
  },
  "assemblies": [
    {
      "assemblyName": "Baseboard",
      "assemblyId": "BB-001",
      "manufacturerSerialNumber": "BB-SN-12345",
      "components": [
        {
          "componentType": "CPU",
          "componentId": "CPU-001",
          "manufacturerSerialNumber": "CPU-SN-11111",
          "firmware": [
            {
              "version": "1.2.3",
              "lastUpdated": "2025-10-01T12:00:00Z",
              "securityVersionNumber": 5,
              "minimumSecurityVersionNumber": 3,
              "imageRole": "Active"
            }
          ]
        },
        {
          "componentType": "CPU",
          "componentId": "CPU-002",
          "manufacturerSerialNumber": "CPU-SN-22222",
          "firmware": [
            {
              "version": "1.2.3",
              "lastUpdated": "2025-10-01T12:00:00Z",
              "securityVersionNumber": 5,
              "minimumSecurityVersionNumber": 3,
              "imageRole": "Active"
            }
          ]
```

```
    },
    {
      "componentType": "Memory",
      "componentId": "MEM-001",
      "manufacturerSerialNumber": "MEM-SN-33333",
      "firmware": [
        { "version": "N/A" }
      ]
    },
    {
      "componentType": "Memory",
      "componentId": "MEM-002",
      "manufacturerSerialNumber": "MEM-SN-44444",
      "firmware": [
        { "version": "N/A" }
      ]
    },
    {
      "componentType": "BMC",
      "componentId": "BMC-001",
      "manufacturerSerialNumber": "BMC-SN-55555",
      "firmware": [
        {
          "version": "3.4.5",
          "lastUpdated": "2025-09-15T08:00:00Z",
          "securityVersionNumber": 2,
          "minimumSecurityVersionNumber": 1,
          "imageRole": "Active"
        }
      ]
    }
  ],
  "subAssemblies": [
    {
      "assemblyName": "HotSwap Backplane",
      "assemblyId": "HSBP-001",
      "manufacturerSerialNumber": "HSBP-SN-66666",
      "components": [
        {
          "componentType": "Storage",
          "componentId": "NVME-001",
          "manufacturerSerialNumber": "NVME-SN-77777",
          "firmware": [
            { "version": "2.1.0" }
          ]
        },
        {
          "componentType": "Storage",
          "componentId": "NVME-002",
          "manufacturerSerialNumber": "NVME-SN-88888",
          "firmware": [
            { "version": "2.1.0" }
          ]
        },
        {
          "componentType": "NIC",
          "domponentId": "NIC-001",
          "manufacturerSerialNumber": "NIC-SN-99999",
          "firmware": [
            { "version": "1.0.0" }
          ]
        },
        {
          "componentType": "NIC",
          "componentId": "NIC-002",
          "manufacturerSerialNumber": "NIC-SN-10101",
          "firmware": [
            { "version": "1.0.0" }
```

```
            ]
          }
        ]
      },
      {
        "assemblyName": "UBB",
        "assemblyId": "UBB-001",
        "manufacturerSerialNumber": "UBB-SN-20202",
        "subAssemblies": [
          {
            "assemblyName": "OAM-1",
            "assemblyId": "OAM-001",
            "manufacturerSerialNumber": "OAM-SN-30303",
            "components": [
              {
                "componentType": "GPU",
                "componentId": "GPU-001",
                "manufacturerSerialNumber": "GPU-SN-40404",
                "firmware": [
                  { "version": "510.85" }
                ]
              },
              {
                "componentType": "AMC",
                "componentId": "AMC-001",
                "manufacturerSerialNumber": "AMC-SN-50505",
                "firmware": [
                  { "version": "1.0.0" }
                ]
              }
            ]
          },
          {
            "assemblyName": "OAM-2",
            "assemblyId": "OAM-002",
            "manufacturerSerialNumber": "OAM-SN-60606",
            "components": [
              {
                "componentType": "GPU",
                "componentId": "GPU-002",
                "manufacturerSerialNumber": "GPU-SN-70707",
                "firmware": [
                  { "version": "510.85" }
                ]
              },
              {
                "componentType": "AMC",
                "componentId": "AMC-002",
                "manufacturerSerialNumber": "AMC-SN-80808",
                "firmware": [
                  { "version": "1.0.0" }
                ]
              }
            ]
          }
        ]
      }
    ]
  }
}
```

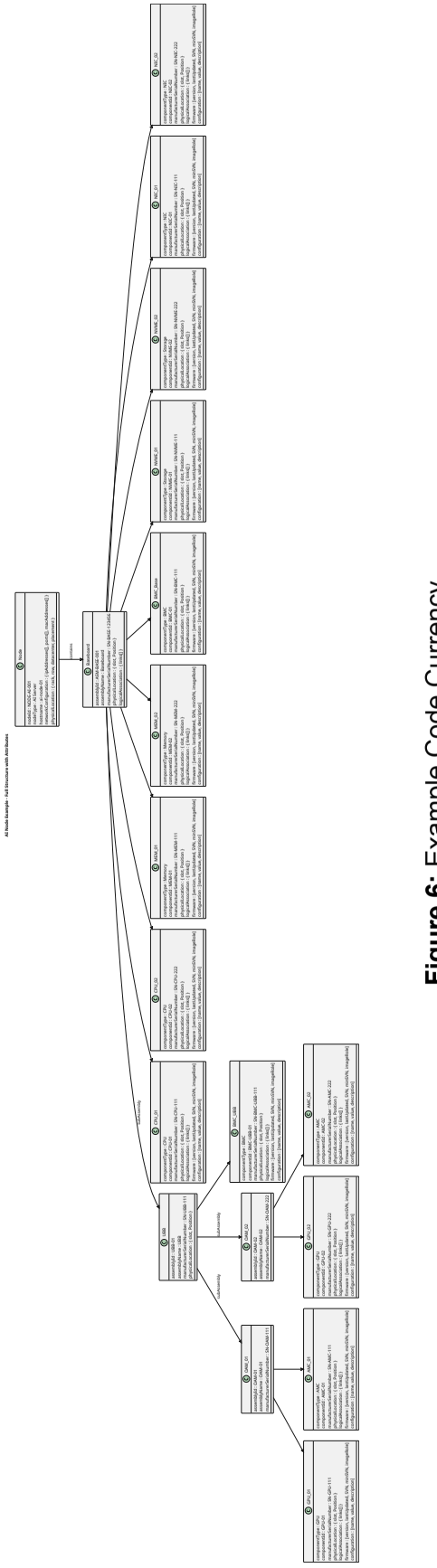

Figure 6: Example Code Currency

Appendix B: Status Dump Examples

References

[1] "OCP Fault Management Infrastructure Requirements." Open Compute Project, Aug. 15, 2025. Available: https://www.opencompute.org/documents/ocp-fault-management-infrastructure-requirements-1-5-pdf

[2] "OCP GPU & Accelerator RAS Requirements." Open Compute Project, Oct. 23, 2025. Available: https://www.opencompute.org/documents/ocp-gpu-and-accelerators-ras-requirements-v1-7-10-23-2025-pdf

[3] "OCP RAS API Specification." Open Compute Project, Dec. 14, 2025. Available: https://www.opencompute.org/documents/2025-ocp-ras-api-v0-9-final-pdf

[4] "OCP Hyperscale CPU Debug and RAS Requirements." Open Compute Project, Sept. 15, 2025. Available: https://www.opencompute.org/documents/hyperscale-cpu-ras-and-debug-requirements-specification-v0-7-09-29-2025-pdf